

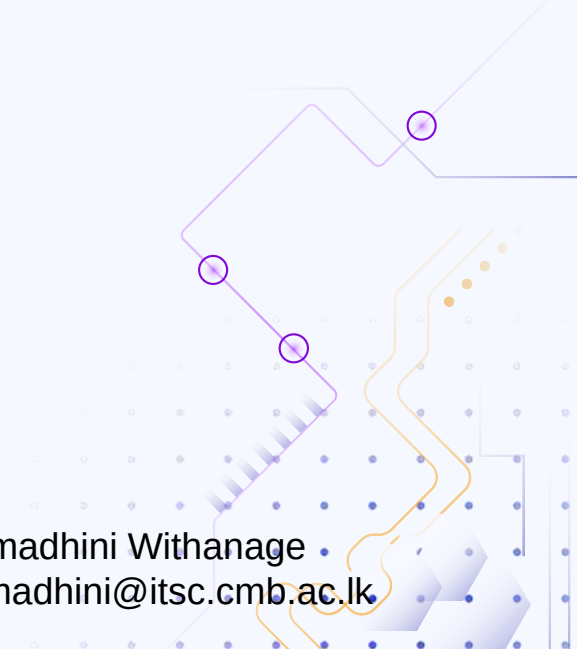


IT 3003

Advanced Programming Techniques

Java Practical Session 2

Start small, think big, and keep coding

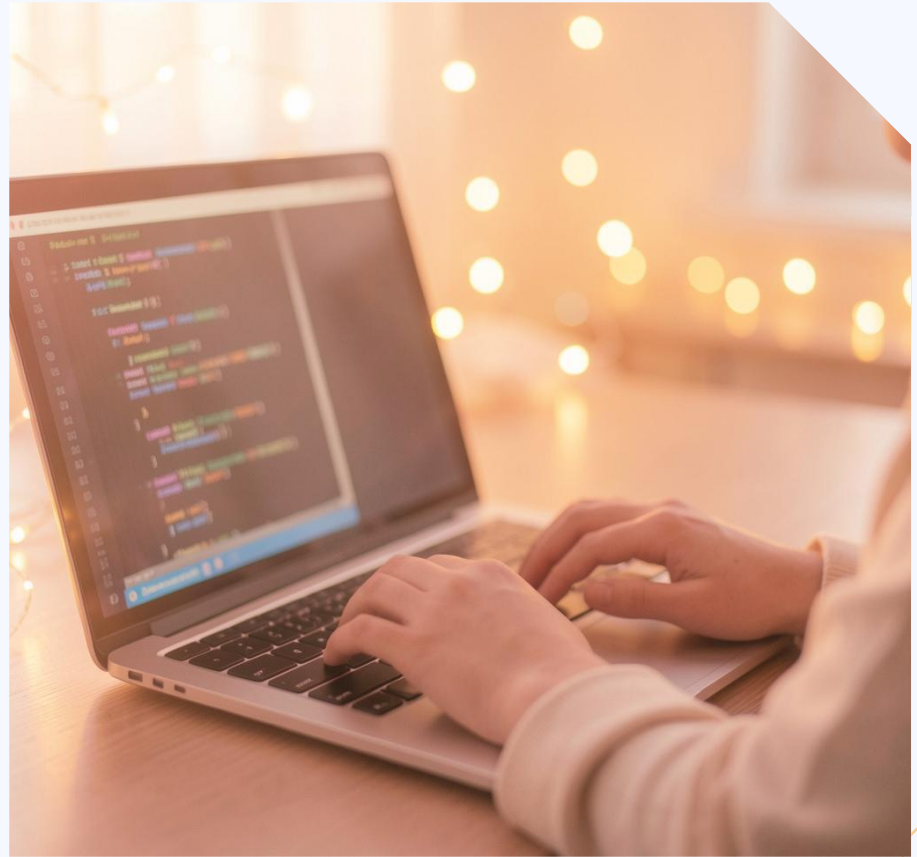


Samadhini Withanage
samadhini@itsc.cmb.ac.lk

Introduction

Programming is the art of giving clear and logical instructions to a computer to solve problems step by step.

Begin with simple concepts, focus on understanding the logic, and build gradually. Always practice, experiment, and refine your code to develop strong and confident programming skills.





Lesson Outline

- ☀ Classes & Objects (Reminder)
- ☀ Constructors
- ☀ Keywords (this , super)
- ☀ Access Modifiers
- ☀ Encapsulation
- ☀ Inheritance
- ☀ Polymorphism
- ☀ Abstraction

Class and Objects

Create a class called **Student** with:

- studentId
- studentName

Create an object and display the values.

Java source file : **Student.java**

Class and Objects

Create a class called **Car** with:

- Brand
- Model
- Year

Create **two objects** and print their details.

Java source file : **Car.java**

Constructors

Create a class called **Employee**.

Create a **constructor** that prints: " Employee Object Created"

Create an object from the class.

Java source file : Employee.java

Constructor Overloading

Create:

1. A constructor without parameters
2. A constructor with brand and price parameters

Display different messages from each constructor.

Java source file : Mobile.java

Constructor Chaining

Create a class Employee with constructors using constructor chaining.

Variables:

- name
- id

Create:

- Default constructor
- Constructor with name
- Constructor with name and id (final constructor)

Use constructor chaining and display values.

Java source file : ConChaining.java

this Keyword

Create a class called **ExThis** with:

- A variable named **name**
- A variable named **age**

Create a parameterized constructor that receives **name and age** as parameters.
Inside the constructor:

- Use the **this keyword** to assign constructor parameter values to instance variables

Create a method called **display()** that prints the values of name and age.
Inside the **main()** method:

- Create an object of the **ExThis** class
- Pass values through the constructor
- Call the **display()** method to demonstrate the use of the this keyword.

Java source file : ExThis.java

super Keyword

Create a parent class called **ExSuper** with:

- A variable named **name** initialized as "**Parent Name**"
- A method named **display()** that prints the parent class name value

Then create a child class called **Teacher** that **extends ExSuper** with:

- Another variable named **name** initialized as "**Teacher Name**"
- An overridden **display()** method

Inside the child class **display()** method:

- Use the **super keyword** to call the parent class **display()** method
- Display the child class variable value

Inside the **main()** method:

- Create an object of the **Teacher** class
- Call the **display()** method to demonstrate the use of the **super keyword**.

Java source file : ExSuper.java

Access Modifiers

Create a class called **TestAccess** with the following variables:

- A **private integer** variable named **a**
- A **protected integer** variable named **b**
- A **public integer** variable named **c**
- A **default integer** variable named **d**

Assign values to all variables.

Create a method called **display()** that prints all variable values.

Inside the **main()** method:

- Create an object of the **TestAccess** class
- Call the **display()** method to demonstrate different access modifiers in Java.

Java source file : TestAccess.java

Encapsulation

Create a class called **BankAccount** with:

- A private variable named **balance**
- A public method named **setBalance()** to assign the balance value
- A public method named **getBalance()** to return the balance value

Inside the **setBalance()** method:

- positive Validate the balance before assigning it
- Only allow values

Inside the **main()** method:

- Create an object of the **BankAccount** class
- Set the balance value
- Display the balance using the **getBalance()** method to demonstrate encapsulation.

Java source file : BankAccount.java

Inheritance

Create a class called **Vehicle** with a method named **show()** that displays the message:
This is a vehicle

Then create a **subclass** called **Car** that **extends** the **Vehicle class** and contains another method named **display()** that displays the message: **This is a car**

Inside the **main()** method:

- Create an object of the **Car** class
- Call both **show()** and **display()** methods to demonstrate inheritance.

Java source file : Vehicle.java

Polymorphism - Method Overloading

Create a class called **Calculator** with multiple methods named **add()** to demonstrate method overloading.

Include the following methods:

- A method to add two integer values
- A method to add three integer values
- A method to add two double values

Inside the **main()** method:

- Create an object of the **Calculator** class
- Call all overloaded **add()** methods
- Display the returned results using **System.out.println()**.

Java source file : Calculator.java

Polymorphism - Method Overriding

Create a class called **Shape** with a method named **draw()** that displays the message: **Drawing shape**

Then create a subclass called **Circle** that extends **Shape** and overrides the **draw()** method to display the message: **Drawing circle**

Inside the **main()** method:

- Create an object using parent class reference
- Assign a **Circle** object to it
- Call the **draw()** method to demonstrate method overriding.

Java source file : Shape.java

Abstraction - Abstract Class

Create an abstract class called **ExAbsClass** with:

- An abstract method named **calculateSalary()**
- A normal method named **show()** that displays the message: **Employee details**

Then create a subclass called **FullTimeEmployee** that extends **ExAbsClass** and implements the **calculateSalary()** method by displaying the message: **Salary calculated**

Inside the **main()** method:

- Create an object of **FullTimeEmployee**
- Call both **calculateSalary()** and **show()** methods.

Java source file : ExAbsClass.java

Abstraction - Interface

Create an interface called **Payment** with an **abstract method** named **pay()**.

Then create a class called **CreditCard** that **implements the Payment interface** and provides the **implementation for the pay() method** by displaying the message: **Paid using Credit Card**

Inside the main() method:

- Create an object using interface reference
- Call the pay() method to display the payment message.

Java source file : Payment.java

Take-Home Exercise

Lab Sheet 02



Thanks!

